Parcial Algoritmos y Programación II - 7541 Curso Calvo (3º instancia) 1 de Julio de 2014

Padrón: **Apellido y Nombre:**

Punteros: APROBADO - DESAPROBADO

1) Indicar la salida por pantalla y escribir las sentencias necesarias para liberar correctamente la memoria.

```
int main(){

  int *A, *C, *F;
  int **B;
  char *D, *E;
  char G;
  int H;

  H = 65;
  G = (char) H;
  A = new int;
  (*A) = H;
  F = A;
  C = F;
  (*A) = 67;
  cout << (*A) << (*C) << (*F) << endl;

  B = &C;
  C = new int;
  (**B) = H + 1;
  cout << (*A) << (*C) << (**B) << endl;

  E = (char*) C;
  D = (char*) (*B);
  cout << (*E) << (*D) << endl;

  (*E) = G;
  cout << (*E) << (*C) << endl;

  F = (int*) D;
  while ( (*C) > 0) {
      (*C)--;
      cout << (*C) << (*D) << (**B) << endl;
      (*F) = (*F) - (**B);
  }

  // liberar la memoria
  return 0;
}
```

2. Implementar para clase Cola<T> las primitivas, indicando pre y post condiciones:

```
void acolar(T elemento);
T desacolar();
```

3. Implementar el método `determinarCantidadResultanteDelElemento` de la clase `Caminante` a partir de las siguientes especificaciones:

```
class Caminante {
  public:
    /* post: procesa 'recorridos' para determinar la cantidad de total de Elementos identificados por 'nombreElemento'
     *       que fueron levantados y no fueron tirados durante cada Recorrido.
     *       Devuelve la suma de las cantidades conservadas para el Elemento.
     */
    unsigned int determinarCantidadResultanteDelElemento(Lista<Recorrido*>* recorridos, string nombreElemento);
};
```

```
class Recorrido {

  public:

    /* post: inicia el Recorrido con el nombre indicado, sin
     *       Elementos asociados.
     */
    Recorrido(string nombre);

    /* post: elimina todos los Elementos asociados.
     */
    ~Recorrido();

    /* post: devuelve el nombre que identifica al Recorrido.
     */
    string obtenerNombre();

    /* post: devuelve todos los Elementos levantados durante el Recorrido.
     */
    Lista<Elemento*>* obtenerElementosLevantados();

    /* post: devuelve todos los Elementos tirados durante el Recorrido.
     */
    Lista<Elemento*>* obtenerElementosTirados();
};
```

```
class Elemento {

  public:

    /* post: queda inicializado con el nombre y
     *       cantidad en 1.
     */
    Elemento(string nombre);

    ~Elemento();

    /* post: devuelve el nombre que lo distingue
     */
    string obtenerNombre();

    /* post: devuelve la cantidad de unidades.
     */
    unsigned int obtenerCantidad();

    /* post: cambia la cantidad de unidades.
     */
    void cambiarCantidad(unsigned int cantidad);

};
```

4. Diseñar la especificación e implementar el TDA **PingPong**. Debe proveer operaciones para:

- crearlo a partir de los nombres de los jugadores.
- anotar un punto para uno de los jugadores.
- devolver la cantidad de puntos de uno de los jugadores.
- devolver el nombre del ganador.

El juego se acaba cuando uno de los dos jugadores llegue a 7 puntos y el otro no tenga puntos (7-0), o cuando uno de los dos jugadores llega a 11 puntos, ganando por al menos dos puntos de diferencia. En caso de que queden empatados a 10, se seguirán jugando puntos sucesivos hasta que uno de los dos gane por diferencia de dos.

Los alumnos que tienen aprobado el parcialito de punteros no deben realizar el ejercicio 1.
Para aprobar es necesario tener al menos el 60% de cada uno de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Duración del examen : 3 horas